Sistema de Prontuário e Filas UBS/UPA

TDE 2 — Backend em **Vertical Slice**, **Clean Architecture** e **SOLID**
Disciplina de Cloud · 6º Semestre

Alunos e contribuições

Aluno	Papel	O que realizou
Ana Paula Alves	Montagem do documento	Montou este documento de entrega: organização das seções, revisão do texto, seleção dos diagramas e legendas das figuras.
Bernardo Plottegher	Montagem de prompts	Escreveu os prompts a partir da documentação arc42 (requisitos RF01–RF18, matriz Requisito × Endpoint, máquina de estados, eventos e stack) e os prompts de correção: o erro de renderização do diagrama e a reorganização em Vertical Slice com tradução para o português.
Luiz Fernando Grenteski	Execução e GitHub	Executou os prompts no assistente de IA (Claude Code), validou o resultado (renderização dos diagramas, geração das imagens, checagem do código) e publicou o projeto na branch do GitHub.

Repositório GitHub

Repositório	https://github.com/fergrenteski/open-prontuario
Branch	`feature/vertical-slice-clean-architecture` https://github.com/fergrenteski/open-prontuario/tree/feature/vertical-slice-clean-architecture
Diagramas (Markdown)	`docs/arquitetura-vertical-slice.md` https://github.com/fergrenteski/open-prontuario/blob/feature/vertical-slice-clean-architecture/docs/arquitetura-vertical-slice.md
Diagramas (imagem)	`docs/img/arquitetura-1.png` … `arquitetura-12.png`

1. Prompts utilizados

Prompts enviados ao assistente de IA, na ordem e na íntegra. Abaixo de cada um, o resultado obtido.

Prompt 1 — Criação do repositório mínimo e dos diagramas

```
[Anexo: arc42_prontuario_filas.md]

Você é um arquiteto de software especialista em Vertical Slice Architecture, Clean Architecture e
SOLID.

## Contexto
Em anexo está a documentação arc42 do "Sistema de Prontuário e Filas UBS/UPA" (TDE 2). Ainda NÃO
existe código. Use o documento como fonte única da verdade: requisitos RF01–RF18, matriz Requisito ×
Endpoint, diagrama de classes da seção 5.3, máquina de estados da seção 6.3, eventos da seção 8 e a
stack (Spring Boot/Java 21, Cosmos DB API MongoDB, Azure Service Bus, Microsoft Entra ID, Azure
Maps).

## Objetivo
Criar um repositório GitHub MÍNIMO que represente a arquitetura VERTICAL SLICE do backend,
organizado nas 4 camadas da Clean Architecture, com os diagramas de classes e de componentes. O foco
é a ESTRUTURA, não a implementação.

As 4 camadas que cada fatia atravessa:
1. Entities: regras de negócio puras (RegistroFila, Consentimento, Prontuario, LogAuditoria,
UnidadeSaude, VisaoAgregada + enums)
2. Use Cases: um handler por fatia + portas (interfaces) que ele usa
3. Interface Adapters: controllers REST, DTOs Request/Response, implementações de repositório e
publisher de eventos
4. Frameworks & Drivers: Spring Boot, Spring Data MongoDB/Cosmos DB, Azure Service Bus, Entra ID,
Azure Maps

Fatias (uma por endpoint da matriz de rastreabilidade):
- Fila: AvisarChegada, ListarFilaPorPrioridade, ConfirmarChegada, AlterarPrioridade,
IniciarAtendimento, RemoverDaFila
- Unidade: ListarUnidadesProximas, ListarMinhasUnidades, ConsultarVisaoAgregada
- Consentimento: ConcederConsentimento, RevogarConsentimento, ListarConsentimentos
- Prontuário: ConsultarProprioProntuario, ConsultarProntuario
- Auditoria: ListarAuditoriaProntuario
- Agregação (Worker): AtualizarVisaoAgregada

## Parte 1 — Repositório mínimo
Estrutura de pastas:

src/main/java/br/edu/tde/prontuariofilas/
  shared/                     (UseCase<I,O>, exceções, UsuarioAtual)
  features/
    fila/
      domain/                 (Entities + portas de repositório do módulo)
      avisarchegada/          (AvisarChegadaCommand, AvisarChegadaHandler, AvisarChegadaEndpoint,
AvisarChegadaRequest/Response)
      confirmarchegada/
      ...                     (uma pasta por fatia)
      infrastructure/         (adapters: MongoRegistroFilaRepository,
ServiceBusEventoFilaPublisher)
    unidade/ ...
    consentimento/ ...
    prontuario/ ...
    auditoria/ ...
    agregacao/ ...

Regras do código mínimo:
- Classes e interfaces com APENAS assinaturas, atributos e anotações. Corpos de método vazios ou
`throw new UnsupportedOperationException("TODO")`.
- Exceção: implementar por completo UMA fatia de exemplo (AvisarChegada), para mostrar o fluxo
```

```
Endpoint → Handler → Entity → Porta → Adapter.
- Entidades sem anotações de Spring/Mongo; handlers dependem só de interfaces (DIP).
- Incluir: pom.xml (Spring Boot 3.3, Java 21), application.yml básico, .gitignore e README.md.
- README.md com: objetivo, explicação curta de Vertical Slice + Clean Architecture, árvore de
pastas, tabela "Princípio SOLID → onde aparece" e link para docs/.

## Parte 2 — Diagramas (Mermaid, em docs/arquitetura-vertical-slice.md)
1. Diagrama de Classes (classDiagram):
   - `namespace` por fatia/módulo; estereótipos <<Entity>>, <<UseCase>>, <<Port>>, <<Adapter>>,
<<Framework>>.
   - Entidades com atributos/métodos do arc42; handlers com executar(); portas e adapters que as
implementam (..|>); dependências (..>) sempre apontando para dentro.
   - Se ficar ilegível, um diagrama por módulo + um resumo das entidades.
2. Diagrama de Componentes (flowchart):
   - Cada fatia como subgraph "«slice» Nome" com Endpoint → UseCase → Porta.
   - Camadas Entities / Use Cases / Adapters / Frameworks visíveis, cortadas pelas fatias.
   - Interfaces providas/requeridas ({{"«interface» ..."}}) e externos: Painel Web, App Mobile,
Cosmos DB, Service Bus, Worker (Azure Functions), Entra ID, Azure Maps.
3. Os diagramas devem bater exatamente com as classes e pastas do repositório.
4. Gerar as imagens em docs/img/:
   npx @mermaid-js/mermaid-cli -i docs/arquitetura-vertical-slice.md -o docs/img/arquitetura.png

## Parte 3 — Git
Comandos para: git init, criar a branch feature/vertical-slice-clean-architecture, commit e push
para o GitHub.
```

> **Resultado:** Gerou o projeto Spring Boot (pom.xml, application.yml, .gitignore, README), as 16 fatias com assinaturas, a fatia AvisarChegada implementada de ponta a ponta e os diagramas Mermaid de classes e componentes.

Prompt 2 — Correção de erro de renderização do diagrama 1.2

````
1.2 Módulo `fila` (6 fatias)

```
Cannot set properties of undefined (setting 'order')
```
````

> **Resultado:** O diagrama de classes do módulo fila não renderizava no Mermaid 11.4 (bug de layout com muitos namespaces). O diagrama foi dividido em dois, sem perder classes nem relações.

Prompt 3 — Reorganização em Vertical Slice e tradução para português

```
A estrutura que você gerou está organizada em 4 camadas (pastas por camada), e isso não é Vertical
Slice. Ajuste o que já foi feito, sem refazer do zero:

1. Remova todas as pastas de camada (domain, application, infrastructure, controllers, services,
repositories, adapters, entities, usecases etc.). Organize por funcionalidade: cada caso de uso vira
uma pasta (fatia) dentro do seu módulo, por exemplo fila/avisarchegada/, fila/confirmarchegada/,
consentimento/concederconsentimento/.

2. Entidade, Caso de Uso, Adaptador e Framework continuam existindo, mas como papéis de arquivo
dentro de cada fatia, e não como pastas. Cada fatia contém: <Fatia>Endpoint, <Fatia>Requisicao,
<Fatia>Resposta, <Fatia>Comando e <Fatia>CasoDeUso.

3. Entidades e portas usadas por mais de uma fatia ficam soltas na raiz do módulo (ex.:
fila/RegistroFila.java, fila/RepositorioRegistroFila.java). Uma fatia nunca importa classes de outra
fatia.

4. Traduza tudo para português: pastas, pacotes, classes, métodos, atributos, comentários, README e
diagramas (ex.: UseCase → CasoDeUso, Handler → CasoDeUso, Controller → Endpoint, Request →
```

```
Requisicao, Response → Resposta, Repository → Repositorio).

5. Refaça os diagramas Mermaid agrupando por fatia (namespace/subgraph por fatia, com estereótipos
<<Entidade>>, <<CasoDeUso>>, <<Porta>>, <<Adaptador>>, <<Framework>>). No diagrama de componentes,
cada fatia deve aparecer como uma coluna vertical.

Responda apenas com: a nova árvore de pastas, os arquivos que mudaram (completos) e os diagramas
atualizados.
```

> **Resultado:** Removeu as pastas de camada; cada caso de uso virou uma pasta (fatia) com Endpoint, Requisicao, Resposta, Comando e CasoDeUso; entidades/portas compartilhadas foram para a raiz do módulo; tudo traduzido; diagramas refeitos por fatia, com cada fatia como coluna no diagrama de componentes.

Prompt 4 — Geração deste documento de entrega

```
Agora gere o PDF com os seguintes itens:

No arquivo deve constar o nome dos ALUNOS que auxiliou na tarefa, independente de ser atividade em
grupo. (Descreva o que cada aluno realizou).

Ana Paula (Montagem de Documento)
Bernardo Plottegher (Montagem de prompts)
Luiz Fernando (Execucao e github)

Informar o GITHUB do projeto em uma nova branch.
https://github.com/fergrenteski/open-prontuario

Informar todos os prompts utilizados para modificar a aplicação.

Entregar diagrama de classes e componentes do BACKEND da aplicação em VERTICAL SLICE e CLEAN
ARCHITECTURE e SOLID (gerar em markdown e imagem).
```

> **Resultado:** Gerou este PDF com alunos, link do GitHub, prompts e diagramas.

2. Arquitetura do backend

Vertical Slice: o código é organizado por **funcionalidade**, não por camada técnica. Cada operação da API (uma linha da matriz Requisito × Endpoint do arc42) é uma pasta — a **fatia** — com tudo de que precisa. Não existem pastas `domain`, `infrastructure`, `controllers` ou `services`. Entidades, portas e adaptadores usados por mais de uma fatia ficam na raiz do módulo, e **uma fatia nunca importa classes de outra fatia**.

Clean Architecture: as 4 camadas continuam existindo como **papéis de arquivo**, e as dependências apontam sempre para dentro:

Estereótipo	Arquivos	Camada
«Adaptador»	`<Fatia>Endpoint`, `<Fatia>Requisicao`, `<Fatia>Resposta`; `Repositorio*Mongo`, `PublicadorEventoFilaServiceBus`, `GatewayGeolocalizacaoAzureMaps`	Interface Adapters
«CasoDeUso»	`<Fatia>Comando`, `<Fatia>CasoDeUso`	Use Cases
«Porta»	`CasoDeUso<E,S>` (entrada); `Repositorio*`, `PublicadorEventoFila`, `GatewayGeolocalizacao` (saída)	Use Cases
«Entidade»	`RegistroFila`, `UnidadeSaude`, `VisaoAgregada`, `Consentimento`, `Prontuario`, `Atendimento`, `LogAuditoria` + enums	Entities
«Framework»	`ConfiguracaoSeguranca` (Entra ID), `ConfiguracaoServiceBus`, `RegistroFilaDocumento`, Spring Boot, SDKs Azure	Frameworks & Drivers

Princípios SOLID → onde aparecem

Princípio	Onde aparece
S — Responsabilidade Única	Cada fatia tem um único `CasoDeUso` com um método `executar`; o `Endpoint` só traduz HTTP; `Requisicao`/`Resposta` só carregam dados; os adaptadores só traduzem para Mongo, Service Bus ou Azure Maps.
O — Aberto/Fechado	Nova funcionalidade = nova pasta de fatia, sem alterar as existentes. Trocar Cosmos DB ou Service Bus = novo adaptador para a mesma porta.
L — Substituição de Liskov	Qualquer implementação de `RepositorioRegistroFila` ou `PublicadorEventoFila` (Mongo, em memória, falsa para testes) substitui a outra sem o caso de uso perceber. Todo `<Fatia>CasoDeUso` é intercambiável como `CasoDeUso<E,S>`.
I — Segregação de Interfaces	Portas pequenas e específicas (`PublicadorEventoFila`, `RepositorioProntuario`, `GatewayGeolocalizacao` e `CasoDeUso` têm um método cada), em vez de um `FilaService` com seis operações.
D — Inversão de Dependência	Casos de uso dependem de interfaces (portas), nunca de `MongoTemplate` ou `ServiceBusSenderClient`. Endpoints dependem de `CasoDeUso<E,S>`, não da classe concreta. Entidades não têm anotações de Spring/Mongo.

Árvore de pastas

```
src/main/java/br/edu/tde/prontuariofilas/
├── AplicacaoProntuarioFilas.java
├── compartilhado/        CasoDeUso · UsuarioAtual · exceções · ConfiguracaoSeguranca · ResolvedorUsuarioAtual · TratadorExcecoesApi
└── funcionalidades/
    ├── fila/             RegistroFila + enums · RepositorioRegistroFila · PublicadorEventoFila (+ adaptadores Mongo/Service Bus)
    │   ├── avisarchegada/  ✅  Endpoint · Requisicao · Resposta · Comando · CasoDeUso
    │   ├── listarfilaporprioridade/  confirmarchegada/  alterarprioridade/  iniciaratendimento/
    │   └── removerdafila/            (204, sem Resposta)
    ├── unidade/          UnidadeSaude · TipoUnidade · VisaoAgregada · RepositorioUnidadeSaude · RepositorioVisaoAgregada
    │   ├── listarunidadesproximas/   (+ UnidadeProxima · GatewayGeolocalizacao · GatewayGeolocalizacaoAzureMaps)
    │   ├── listarminhasunidades/
    │   └── consultarvisaoagregada/
    ├── consentimento/    Consentimento · RepositorioConsentimento
    │   ├── concederconsentimento/  revogarconsentimento/ (204)  listarconsentimentos/
    ├── prontuario/       Prontuario · Atendimento · RepositorioProntuario
    │   ├── consultarproprioprontuario/  consultarprontuario/
    ├── auditoria/        LogAuditoria · RepositorioLogAuditoria
    │   └── listarauditoriaprontuario/
    └── agregacao/
        └── atualizarvisaoagregada/   (Worker — Service Bus Trigger)
```

3. Diagramas de classes

Um `namespace` por fatia (ex.: `fila_avisarchegada`) e um pela raiz de cada módulo, com os estereótipos «Entidade», «CasoDeUso», «Porta», «Adaptador» e «Framework». `..|>` = implementa; `..>` = depende. Fonte em Markdown (Mermaid): `docs/arquitetura-vertical-slice.md`.

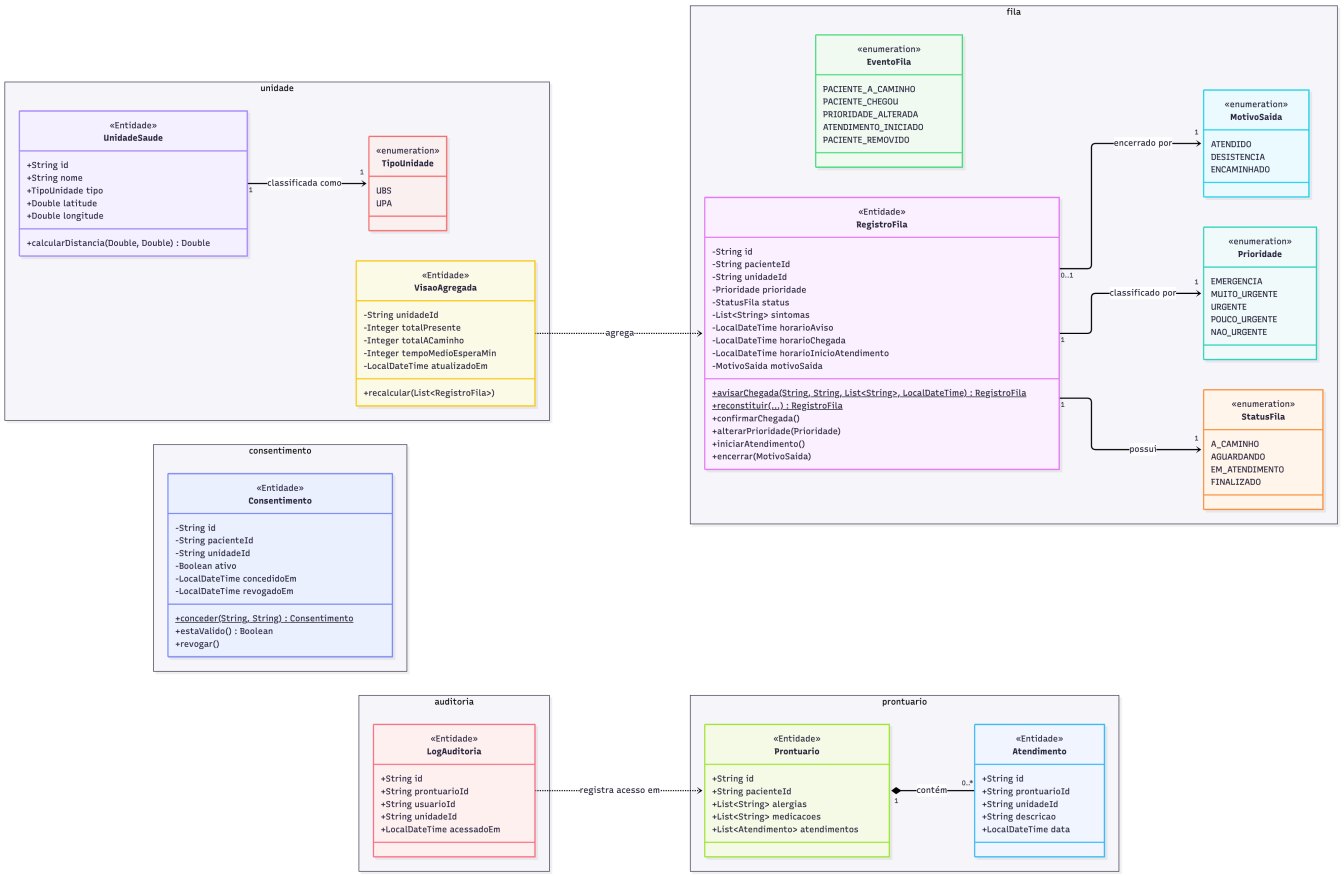


Figura 1 — 1.1 Entidades (raiz de cada módulo)

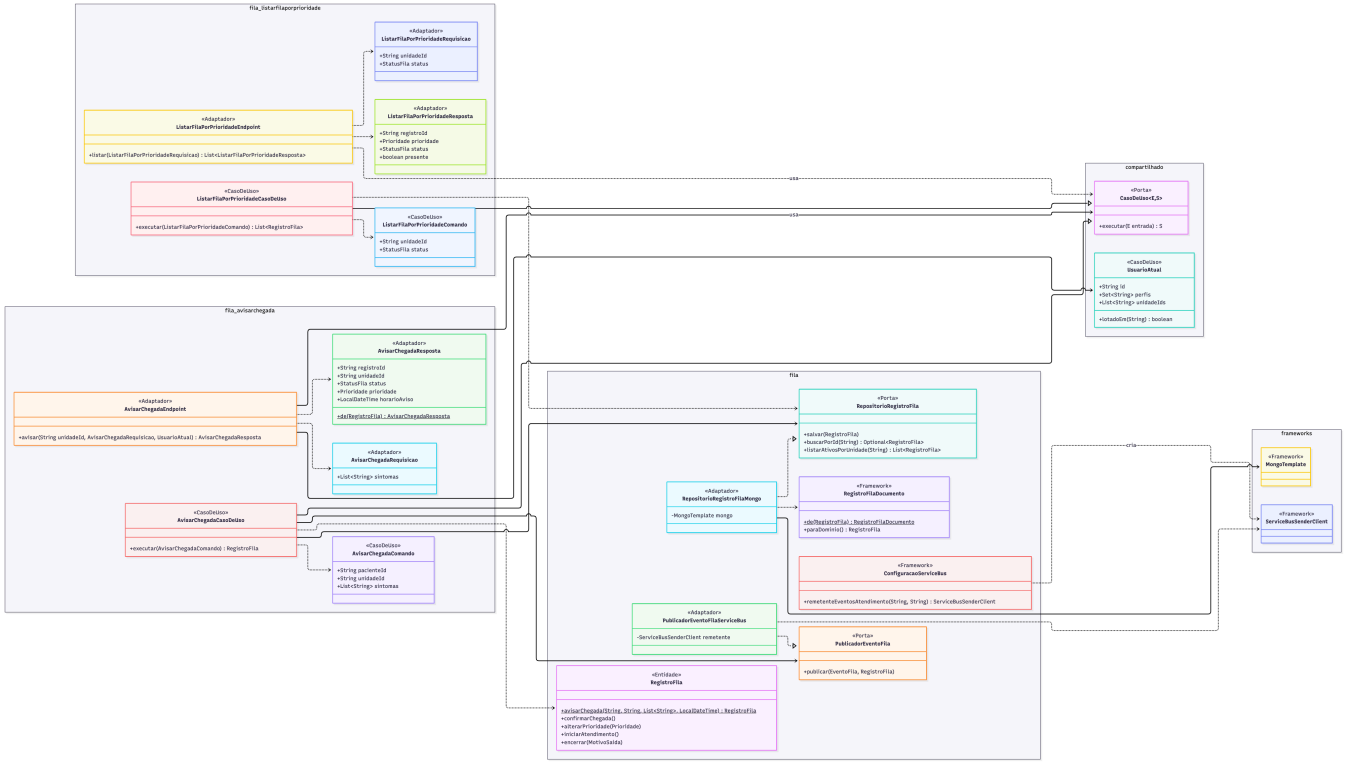


Figura 2 — 1.2 Módulo fila — fatias AvisarChegada (implementada) e ListarFilaPorPrioridade + raiz do módulo

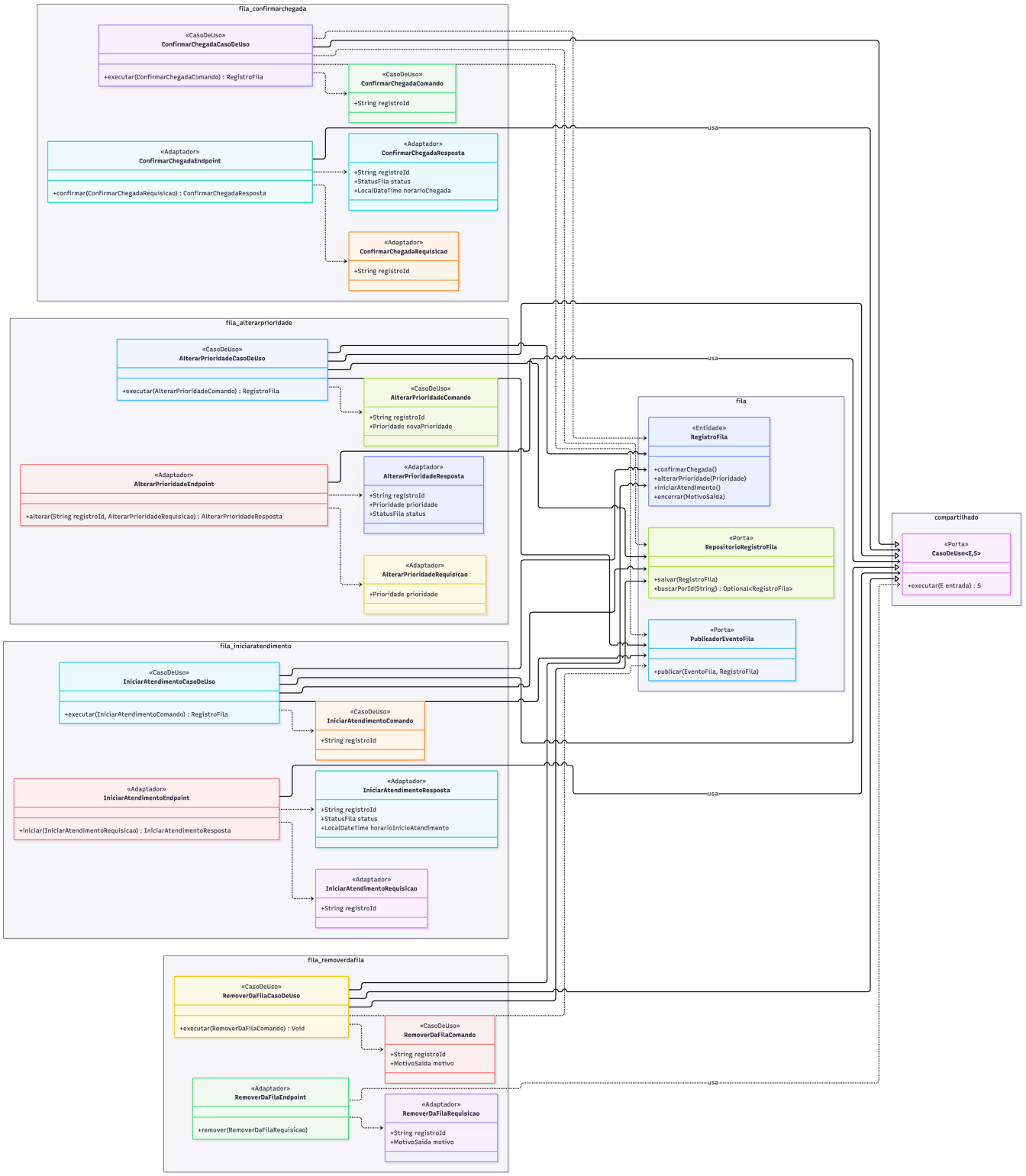


Figura 3 — 1.3 Módulo fila — fatias ConfirmarChegada, AlterarPrioridade, IniciarAtendimento e RemoverDaFila

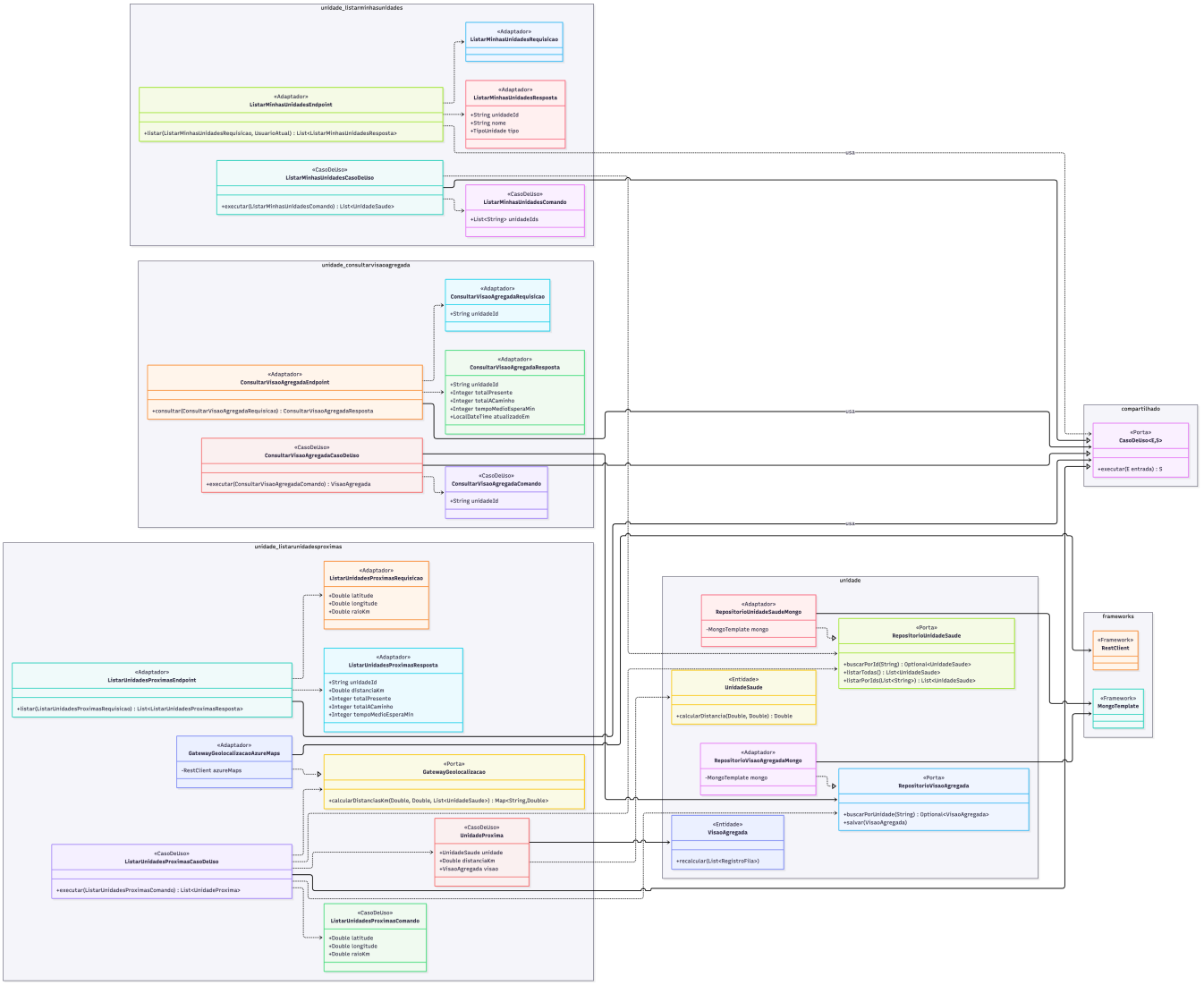


Figura 4 — 1.4 Módulo unidade — fatias ListarUnidadesProximas, ListarMinhasUnidades e ConsultarVisaoAgregada

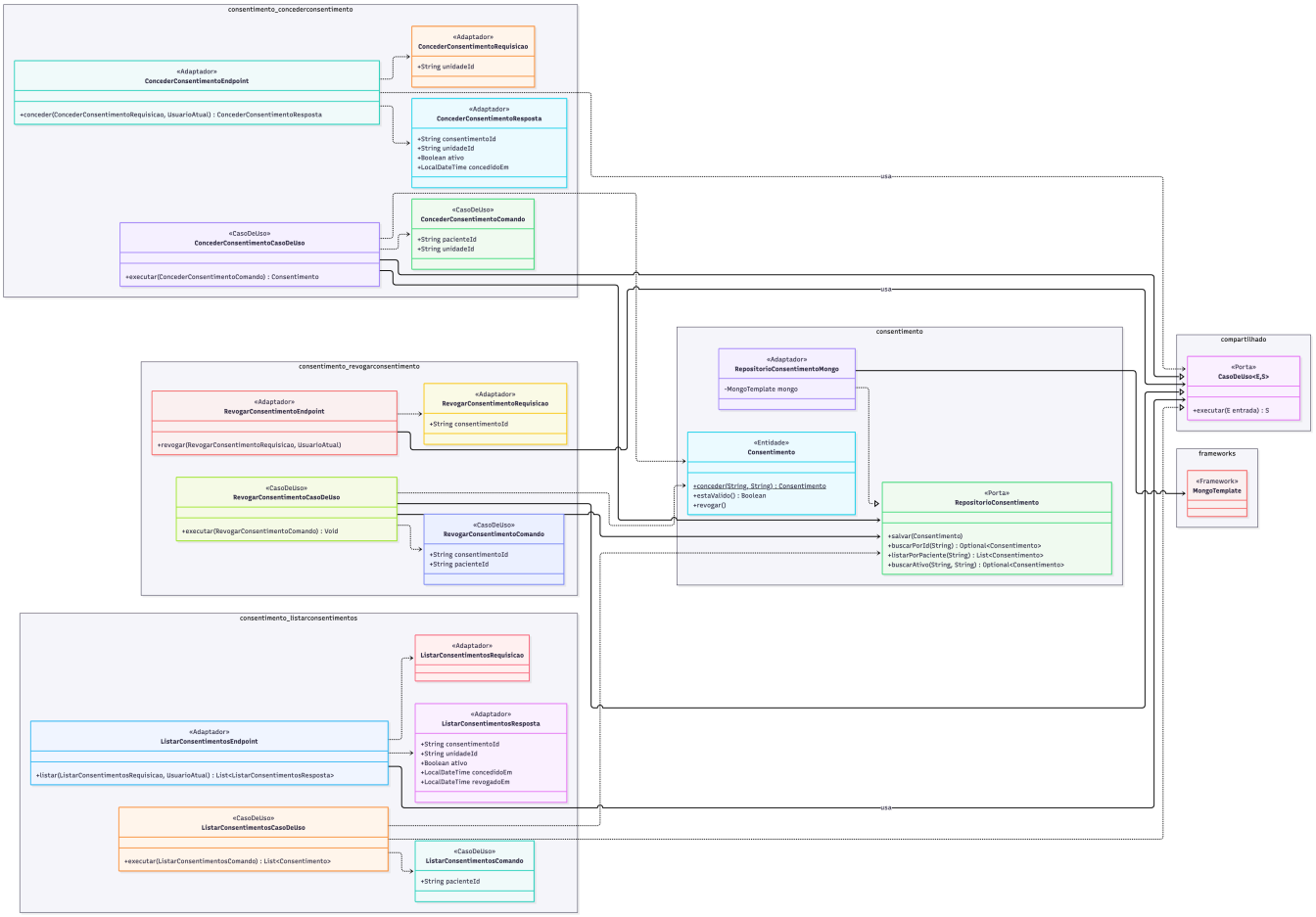


Figura 5 — 1.5 Módulo consentimento — fatias ConcederConsentimento, RevogarConsentimento e ListarConsentimentos

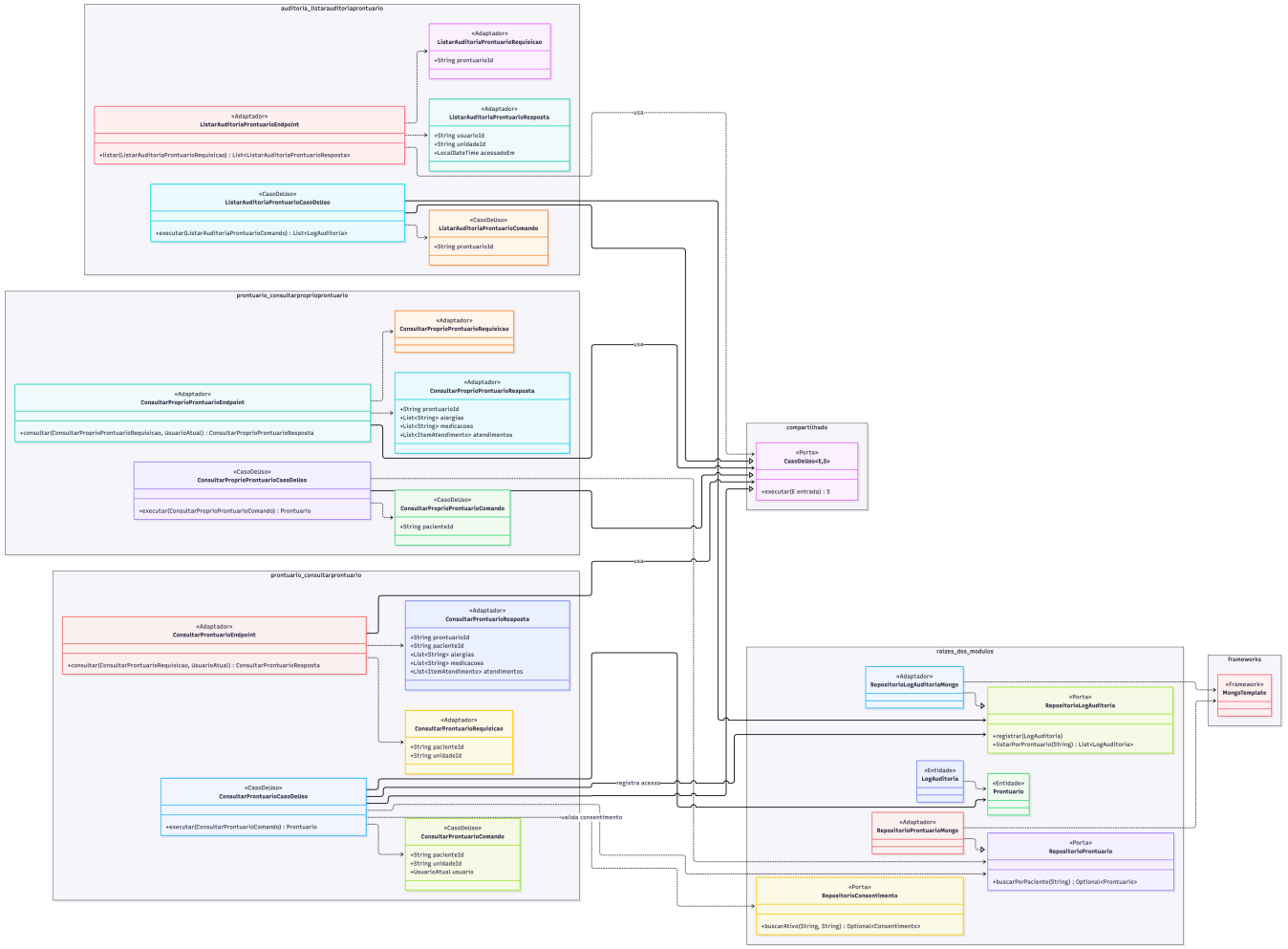


Figura 6 — 1.6 Módulos prontuario e auditoria — fatias ConsultarProprioProntuario, ConsultarProntuario e ListarAuditoriaProntuario

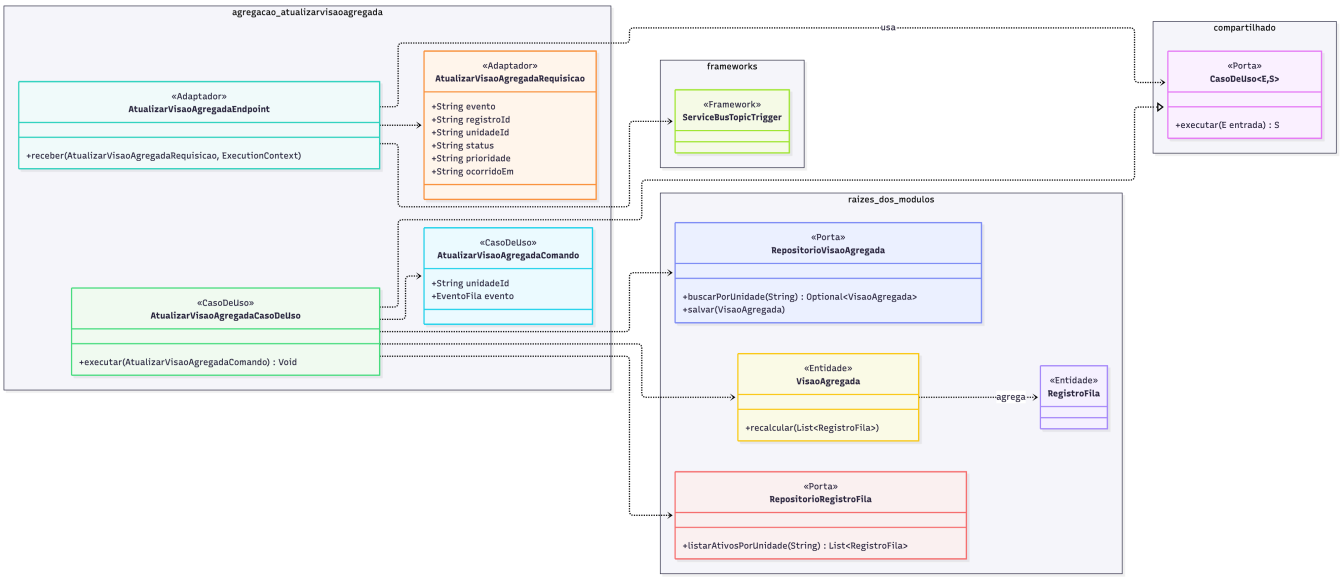


Figura 7 — 1.7 Módulo agregacao (Worker) — fatia AtualizarVisaoAgregada

4. Diagramas de componentes

Cada fatia é uma **coluna vertical**: **Endpoint** (Adaptador) → **«interface» CasoDeUso‹Comando, Saída›** → **CasoDeUso** → portas da raiz do módulo. Abaixo das colunas ficam a raiz do módulo (entidades, portas e adaptadores compartilhados) e os sistemas externos (Painel Web, App Mobile, Cosmos DB, Service Bus, Worker, Entra ID, Azure Maps). As cores indicam o estereótipo (legenda na Figura 8). *requer* = consome a interface; *provê* = implementa.

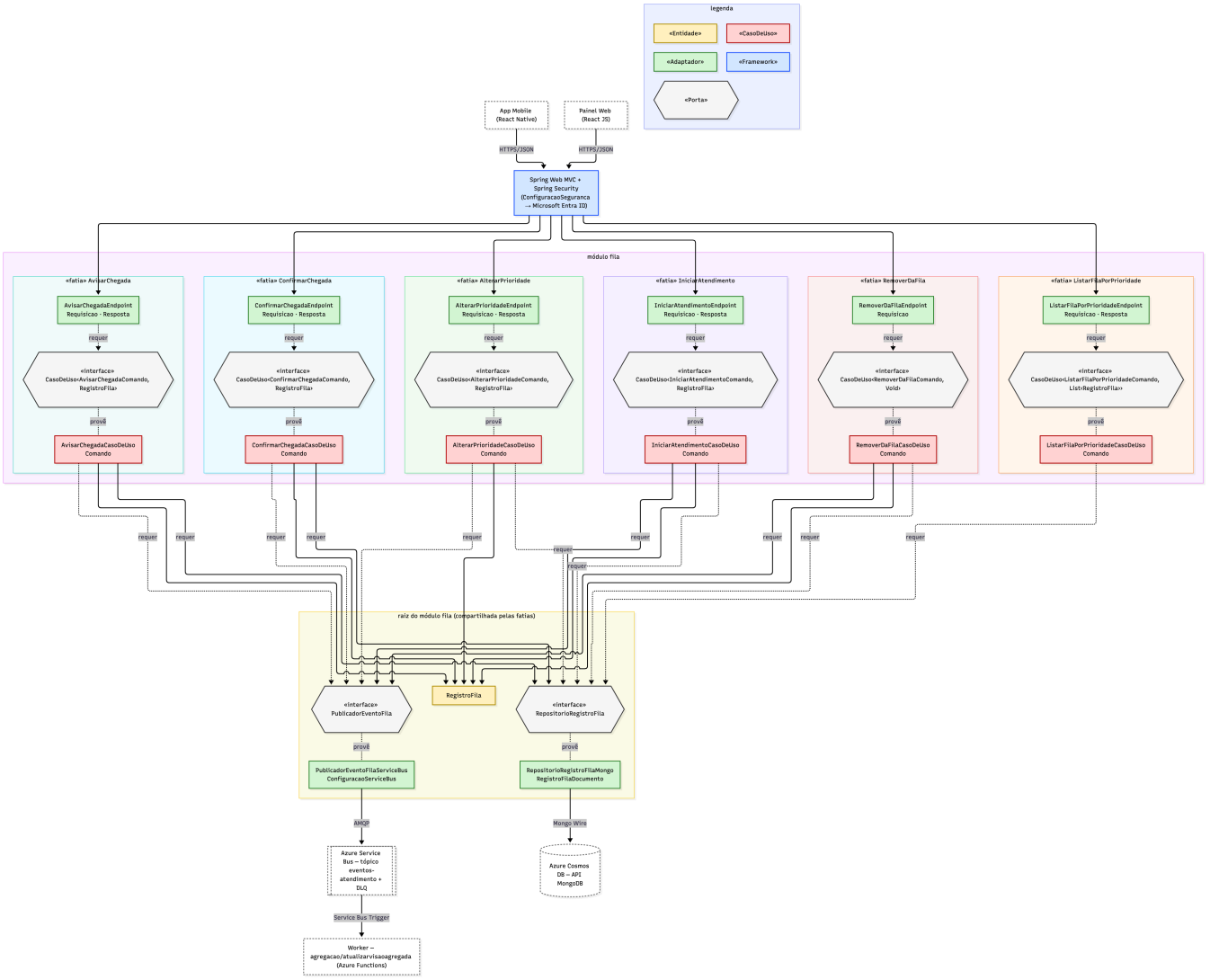


Figura 8 — 2.1 Componentes — módulo fila (uma coluna por fatia)

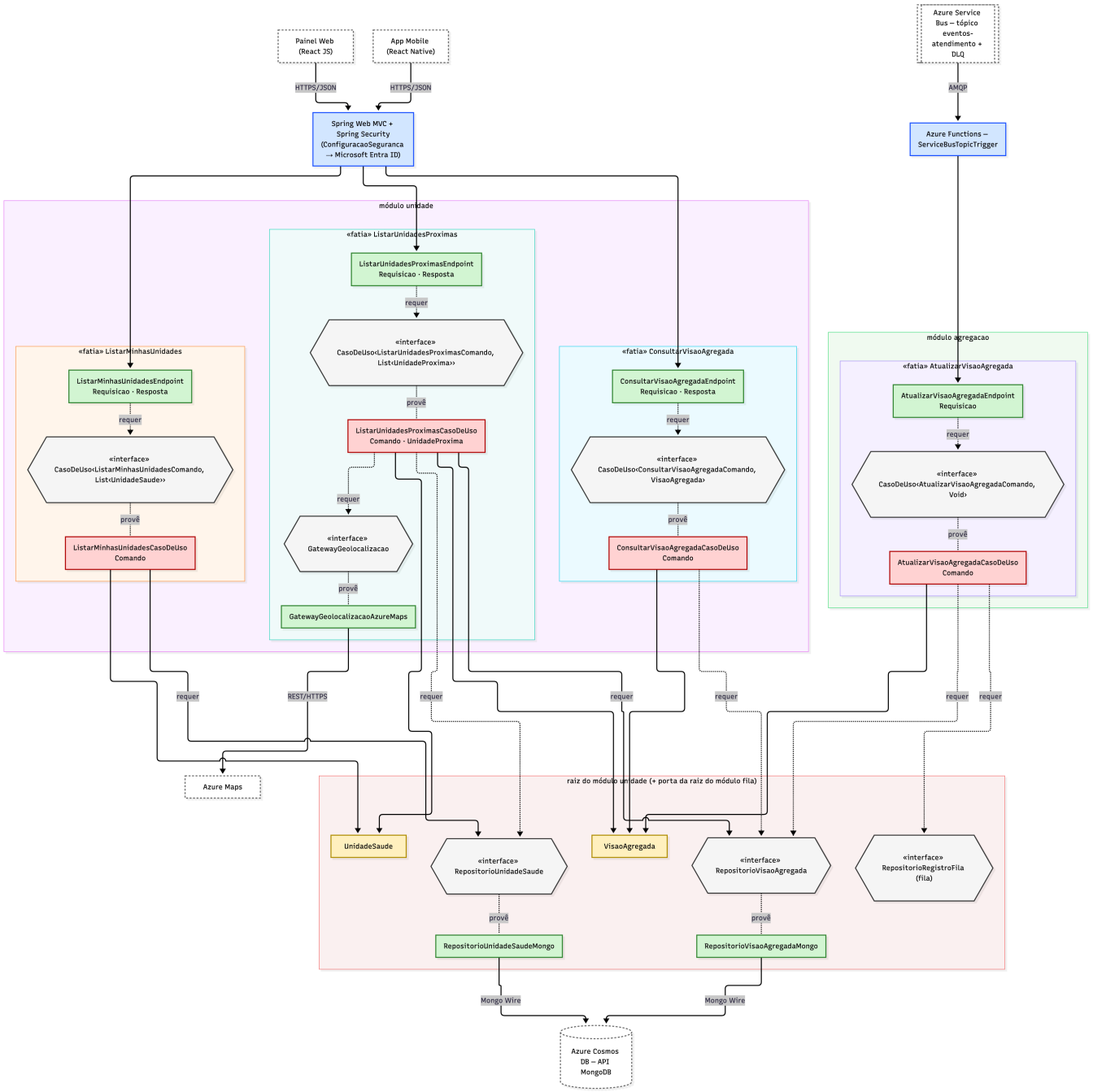


Figura 9 — 2.2 Componentes — módulos unidade e agregacao (Worker)

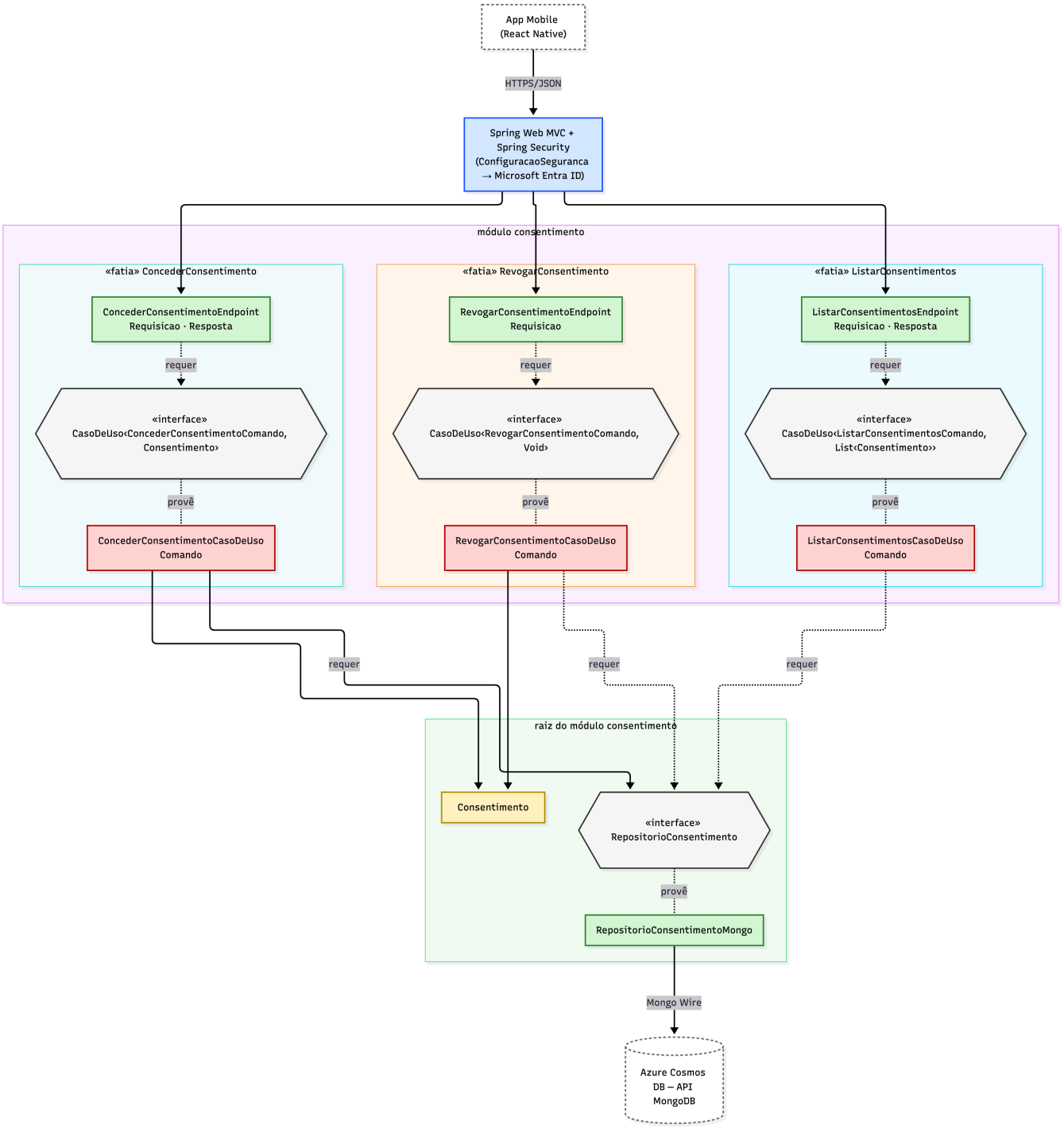


Figura 10 — 2.3 Componentes — módulo consentimento

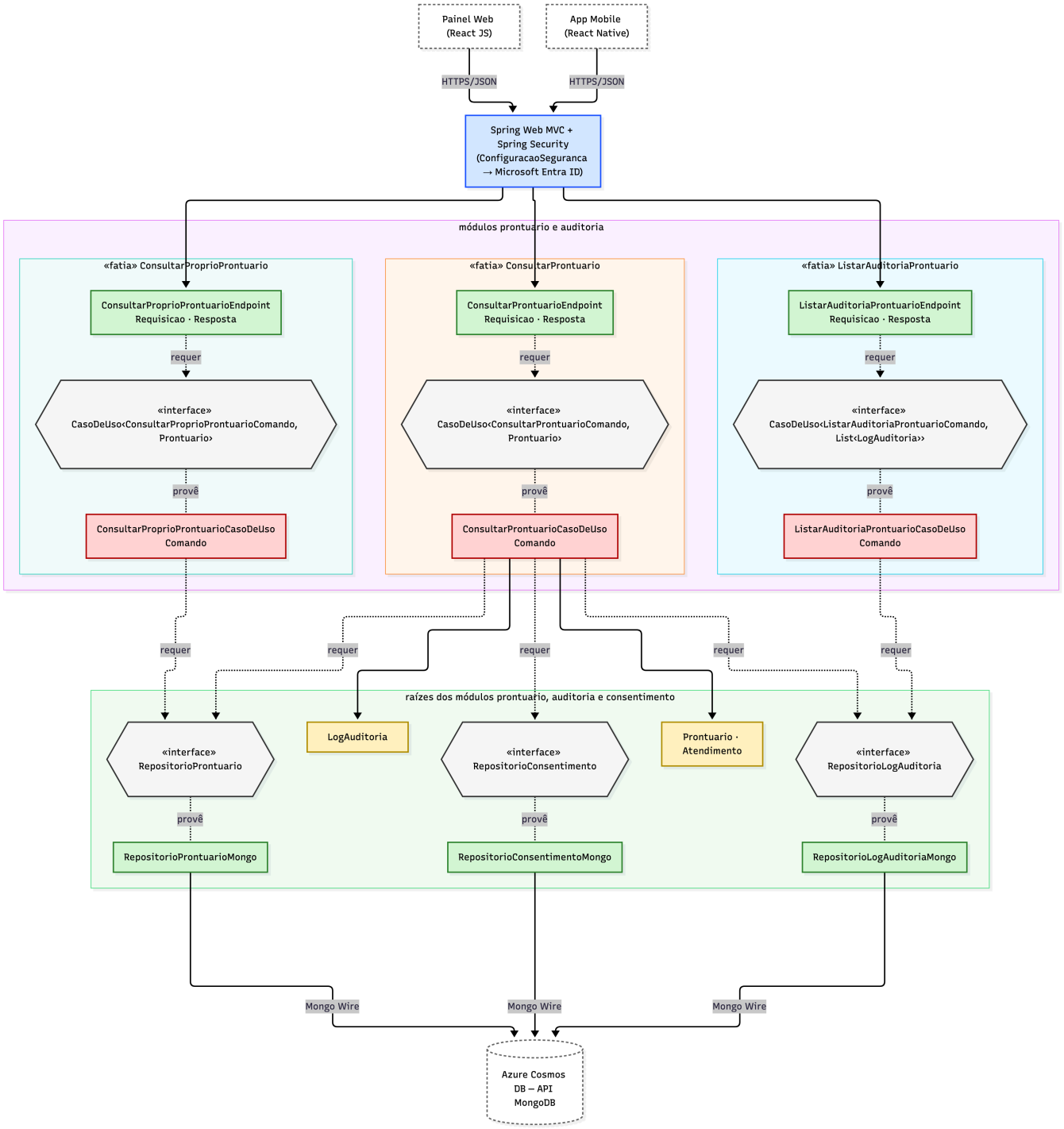


Figura 11 — 2.4 Componentes — módulos prontuario e auditoria

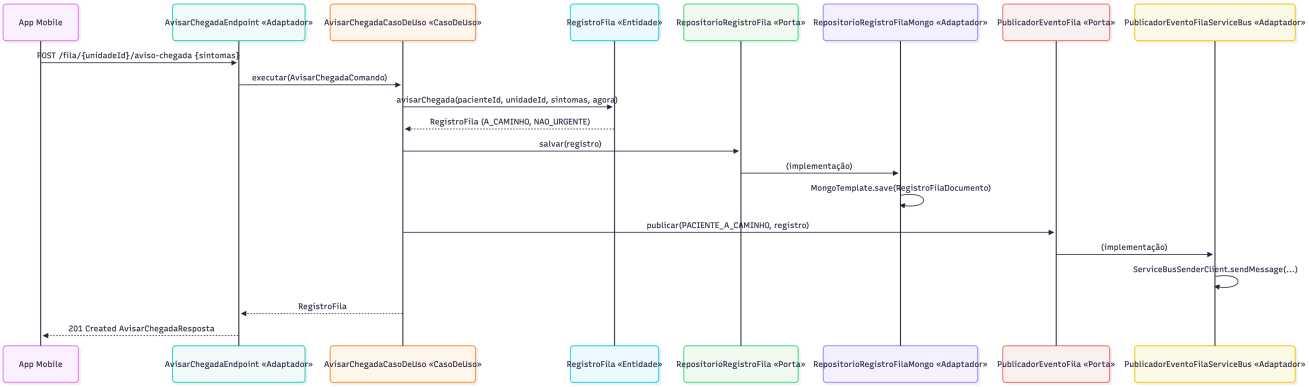


Figura 12 — 3. Sequência — fluxo da fatia AvisarChegada (Endpoint → CasoDeUso → Entidade → Porta → Adaptador)